

Air Quality Index (AQI) Analysis — Delhi

ShadowFox Data Science Internship — Intermediate Task

Objective

Conduct an in-depth analysis of the Air Quality Index (AQI) in Delhi, addressing the specific environmental challenges faced by the city. This notebook defines research questions centered around key pollutants and their behavior, uses statistical analysis and visualization to explore the dynamics of air quality, and computes an India-CPCB-style AQI to interpret the results in terms of public health impact.

Research Questions

1. **Pollutant levels:** What are the levels of the key pollutants (PM2.5, PM10, NO2, SO2, CO, O3, NH3) recorded in Delhi during the observation period, and how do they compare to national air quality standards?
2. **Diurnal patterns:** Does air quality follow a predictable pattern across the hours of the day (e.g. worse during morning/evening traffic hours)?
3. **Pollutant relationships:** Which pollutants are most strongly correlated with each other, and which pollutant most frequently drives the overall AQI value?
4. **Health risk:** How many hours in the observation window fall into "Poor", "Very Poor" or "Severe" AQI categories, and what does that imply for public health advisories?

Dataset & Scope

The dataset (`delhiaqi.csv`) contains **hourly pollutant concentrations for Delhi** covering **1 Jan 2023 to 24 Jan 2023**. It includes 7 pollutants: CO, NO, NO2, O3, SO2, PM2.5, PM10, and NH3, all in ug/m3.

Note on scope: The provided dataset spans about three and a half weeks in January 2023 from a single monitoring point, so this notebook cannot compare *across seasons* or *across geographic zones* of Delhi (both would require a multi-season, multi-station dataset). Instead, the analysis focuses on what the data *can* answer robustly: pollutant levels, diurnal (hour-of-day) patterns, and inter-

pollutant relationships during a winter period -- historically the smog-heaviest time of year for Delhi. This limitation is revisited in the conclusion.

```
In [1]: import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
from scipy import stats

sns.set_theme(style="whitegrid")
plt.rcParams["figure.dpi"] = 100

df = pd.read_csv("delhiaqi.csv")
df["date"] = pd.to_datetime(df["date"])
df = df.sort_values("date").reset_index(drop=True)

print("Shape:", df.shape)
print("Date range:", df['date'].min(), "to", df['date'].max())
df.head()
```

Shape: (561, 9)

Date range: 2023-01-01 00:00:00 to 2023-01-24 08:00:00

```
Out[1]:
```

	date	co	no	no2	o3	so2	pm2_5	pm10	nh3
0	2023-01-01 00:00:00	1655.58	1.66	39.41	5.90	17.88	169.29	194.64	5.83
1	2023-01-01 01:00:00	1869.20	6.82	42.16	1.99	22.17	182.84	211.08	7.66
2	2023-01-01 02:00:00	2510.07	27.72	43.87	0.02	30.04	220.25	260.68	11.40
3	2023-01-01 03:00:00	3150.94	55.43	44.55	0.85	35.76	252.90	304.12	13.55
4	2023-01-01 04:00:00	3471.37	68.84	45.24	5.45	39.10	266.36	322.80	14.19

1. Data Overview & Cleaning

```
In [2]: df.info()
```

```

<class 'pandas.DataFrame'>
RangeIndex: 561 entries, 0 to 560
Data columns (total 9 columns):
 #   Column      Non-Null Count  Dtype
---  -
 0   date        561 non-null    datetime64[us]
 1   co           561 non-null    float64
 2   no           561 non-null    float64
 3   no2          561 non-null    float64
 4   o3           561 non-null    float64
 5   so2          561 non-null    float64
 6   pm2_5        561 non-null    float64
 7   pm10         561 non-null    float64
 8   nh3          561 non-null    float64
dtypes: datetime64[us](1), float64(8)
memory usage: 39.6 KB

```

```

In [3]: # Check for missing values and duplicates
print("Missing values per column:")
print(df.isna().sum())
print("\nDuplicate rows:", df.duplicated().sum())
print("\nDuplicate timestamps:", df['date'].duplicated().sum())

```

```

Missing values per column:
date      0
co         0
no         0
no2        0
o3         0
so2        0
pm2_5      0
pm10       0
nh3        0
dtype: int64

```

```
Duplicate rows: 0
```

```
Duplicate timestamps: 0
```

```

In [4]: df.describe().T

```

Out[4]:

	count	mean	min	25%	50%	75%	max	
date	561	2023-01-12 16:00:00	2023-01-01 00:00:00	2023-01-06 20:00:00	2023-01-12 16:00:00	2023-01-18 12:00:00	2023-01-24 08:00:00	
co	561.0	3814.94221	654.22	1708.98	2590.18	4432.68	16876.22	322
no	561.0	51.181979	0.0	3.38	13.3	59.01	425.58	8
no2	561.0	75.292496	13.37	44.55	63.75	97.33	263.21	4
o3	561.0	30.141943	0.0	0.07	11.8	47.21	164.51	3
so2	561.0	64.655936	5.25	28.13	47.21	77.25	511.17	
pm2_5	561.0	358.256364	60.1	204.45	301.17	416.65	1310.2	22
pm10	561.0	420.988414	69.08	240.9	340.9	482.57	1499.27	27
nh3	561.0	26.425062	0.63	8.23	14.82	26.35	267.51	3

In [5]:

```
# Add helper time features for later grouping
df["hour"] = df["date"].dt.hour
df["day"] = df["date"].dt.date
df["dayofweek"] = df["date"].dt.day_name()
df["is_weekend"] = df["date"].dt.dayofweek >= 5
df["period"] = pd.cut(
    df["hour"],
    bins=[-1, 5, 11, 16, 20, 23],
    labels=["Late Night (0-5)", "Morning (6-11)", "Afternoon (12-16)",
            "Evening (17-20)", "Night (21-23)"],
)
df.head()
```

Out[5]:

	date	co	no	no2	o3	so2	pm2_5	pm10	nh3	hour	day
0	2023-01-01 00:00:00	1655.58	1.66	39.41	5.90	17.88	169.29	194.64	5.83	0	2023-01-01
1	2023-01-01 01:00:00	1869.20	6.82	42.16	1.99	22.17	182.84	211.08	7.66	1	2023-01-01
2	2023-01-01 02:00:00	2510.07	27.72	43.87	0.02	30.04	220.25	260.68	11.40	2	2023-01-01
3	2023-01-01 03:00:00	3150.94	55.43	44.55	0.85	35.76	252.90	304.12	13.55	3	2023-01-01
4	2023-01-01 04:00:00	3471.37	68.84	45.24	5.45	39.10	266.36	322.80	14.19	4	2023-01-01

2. Computing the National Air Quality Index (AQI)

India's CPCB (Central Pollution Control Board) defines a sub-index for each pollutant using piecewise-linear interpolation between fixed concentration breakpoints, then takes the **maximum** of the sub-indices as the overall AQI (the "worst pollutant" determines the headline number). CO's breakpoints are defined in mg/m3, so it is converted from ug/m3 first.

CPCB officially averages PM2.5/PM10/NO2/SO2/NH3 over 24 hours and CO/O3 over 8 hours before applying these breakpoints. Since this dataset is hourly, each hourly reading is scored as if it were that averaging-period value -- a simplification made explicitly for this exploratory analysis; it is best read as a relative severity indicator rather than an official regulatory AQI reading.

```
In [6]: # CPCB breakpoint tables: (C_low, C_high, I_low, I_high)
BREAKPOINTS = {
    "pm2_5": [(0,30,0,50),(30,60,50,100),(60,90,100,200),(90,120,200,300),
              (120,250,300,400),(250,500,400,500)],
    "pm10": [(0,50,0,50),(50,100,50,100),(100,250,100,200),(250,350,200,300),
              (350,430,300,400),(430,600,400,500)],
    "no2": [(0,40,0,50),(40,80,50,100),(80,180,100,200),(180,280,200,300),
              (280,400,300,400),(400,600,400,500)],
    "so2": [(0,40,0,50),(40,80,50,100),(80,380,100,200),(380,800,200,300),
              (800,1600,300,400),(1600,2000,400,500)],
    "co_mg": [(0,1.0,0,50),(1.0,2.0,50,100),(2.0,10,100,200),(10,17,200,300),
              (17,34,300,400),(34,50,400,500)],
    "o3": [(0,50,0,50),(50,100,50,100),(100,168,100,200),(168,208,200,300),
              (208,748,300,400),(748,1000,400,500)],
    "nh3": [(0,200,0,50),(200,400,50,100),(400,800,100,200),(800,1200,200,300),
              (1200,1800,300,400),(1800,2400,400,500)],
}

def sub_index(conc, table):
    if pd.isna(conc):
        return np.nan
    for c_lo, c_hi, i_lo, i_hi in table:
        if c_lo <= conc <= c_hi:
            return ((i_hi - i_lo) / (c_hi - c_lo)) * (conc - c_lo) + i_lo
    # above the highest defined breakpoint -> cap using the last band's slope
    c_lo, c_hi, i_lo, i_hi = table[-1]
    return ((i_hi - i_lo) / (c_hi - c_lo)) * (conc - c_lo) + i_lo

df["co_mg"] = df["co"] / 1000.0 # ug/m3 -> mg/m3

sub_cols = {}
for pollutant, table in BREAKPOINTS.items():
    col = f"SI_{pollutant}"
    df[col] = df[pollutant].apply(lambda c: sub_index(c, table))
    sub_cols[pollutant] = col

df["AQI"] = df[list(sub_cols.values())].max(axis=1)
df["dominant_pollutant"] = df[list(sub_cols.values())].idxmax(axis=1).str.re
```

```
def categorize(aqi):
    if aqi <= 50: return "Good"
    elif aqi <= 100: return "Satisfactory"
    elif aqi <= 200: return "Moderate"
    elif aqi <= 300: return "Poor"
    elif aqi <= 400: return "Very Poor"
    else: return "Severe"

df["AQI_category"] = df["AQI"].apply(categorize)
df[["date", "pm2_5", "pm10", "AQI", "dominant_pollutant", "AQI_category"]].head(10)
```

Out[6]:

	date	pm2_5	pm10	AQI	dominant_pollutant	AQI_category
0	2023-01-01 00:00:00	169.29	194.64	337.915385	pm2_5	Very Poor
1	2023-01-01 01:00:00	182.84	211.08	348.338462	pm2_5	Very Poor
2	2023-01-01 02:00:00	220.25	260.68	377.115385	pm2_5	Very Poor
3	2023-01-01 03:00:00	252.90	304.12	401.160000	pm2_5	Severe
4	2023-01-01 04:00:00	266.36	322.80	406.544000	pm2_5	Severe
5	2023-01-01 05:00:00	276.54	336.79	410.616000	pm2_5	Severe
6	2023-01-01 06:00:00	295.40	357.07	418.160000	pm2_5	Severe
7	2023-01-01 07:00:00	158.83	182.61	329.869231	pm2_5	Very Poor
8	2023-01-01 08:00:00	134.39	153.47	311.069231	pm2_5	Very Poor
9	2023-01-01 09:00:00	133.22	152.09	310.169231	pm2_5	Very Poor

3. Research Question 1 -- Pollutant Levels vs. Standards

In [7]:

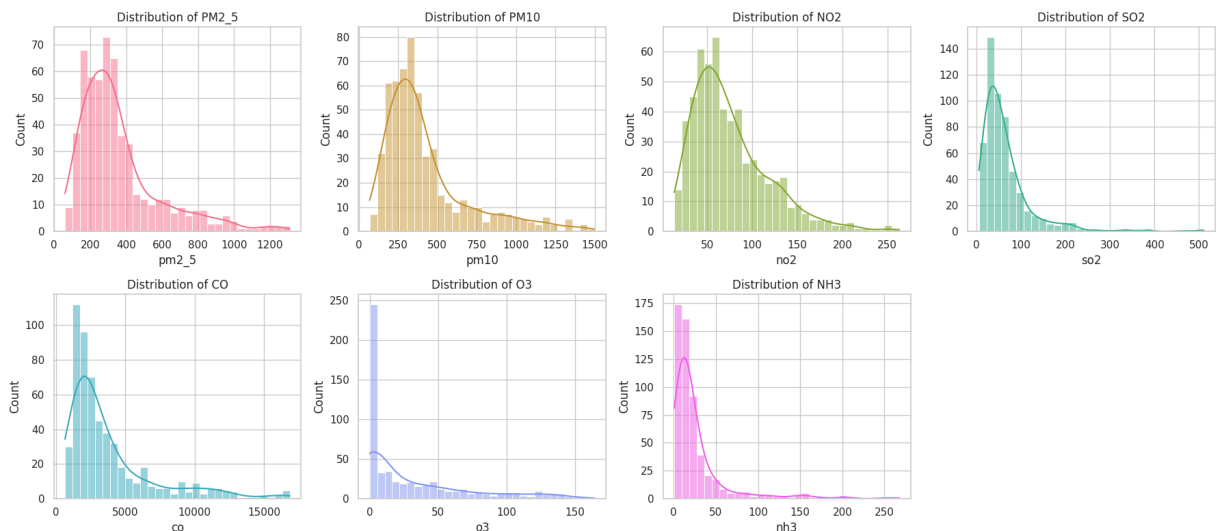
```
pollutant_cols = ["pm2_5", "pm10", "no2", "so2", "co", "o3", "nh3"]
summary = df[pollutant_cols].agg(["mean", "median", "std", "min", "max"]).T
```

```
summary.columns = ["Mean", "Median", "Std Dev", "Min", "Max"]
summary
```

```
Out[7]:
```

	Mean	Median	Std Dev	Min	Max
pm2_5	358.256364	301.17	227.359117	60.10	1310.20
pm10	420.988414	340.90	271.287026	69.08	1499.27
no2	75.292496	63.75	42.473791	13.37	263.21
so2	64.655936	47.21	61.073080	5.25	511.17
co	3814.942210	2590.18	3227.744681	654.22	16876.22
o3	30.141943	11.80	39.979405	0.00	164.51
nh3	26.425062	14.82	36.563094	0.63	267.51

```
In [8]: fig, axes = plt.subplots(2, 4, figsize=(18, 8))
axes = axes.flatten()
for i, col in enumerate(pollutant_cols):
    sns.histplot(df[col], bins=30, kde=True, ax=axes[i], color=sns.color_palette()[i])
    axes[i].set_title(f"Distribution of {col.upper()}")
axes[-1].axis("off")
plt.tight_layout()
plt.savefig("aqi_pollutant_distributions.png", dpi=130)
plt.show()
```



```
In [9]: aqi_counts = df["AQI_category"].value_counts().reindex(
    ["Good", "Satisfactory", "Moderate", "Poor", "Very Poor", "Severe"]
).fillna(0)

plt.figure(figsize=(7, 4.5))
colors = ["#2ecc71", "#a3d977", "#f4d03f", "#e67e22", "#e74c3c", "#7d1e1e"]
sns.barplot(x=aqi_counts.index, y=aqi_counts.values, palette=colors)
plt.ylabel("Number of Hours")
plt.title("Hours Spent in Each AQI Category (Delhi, Jan 2023 sample)")
plt.xticks(rotation=20)
plt.tight_layout()
plt.savefig("aqi_category_counts.png", dpi=130)
```

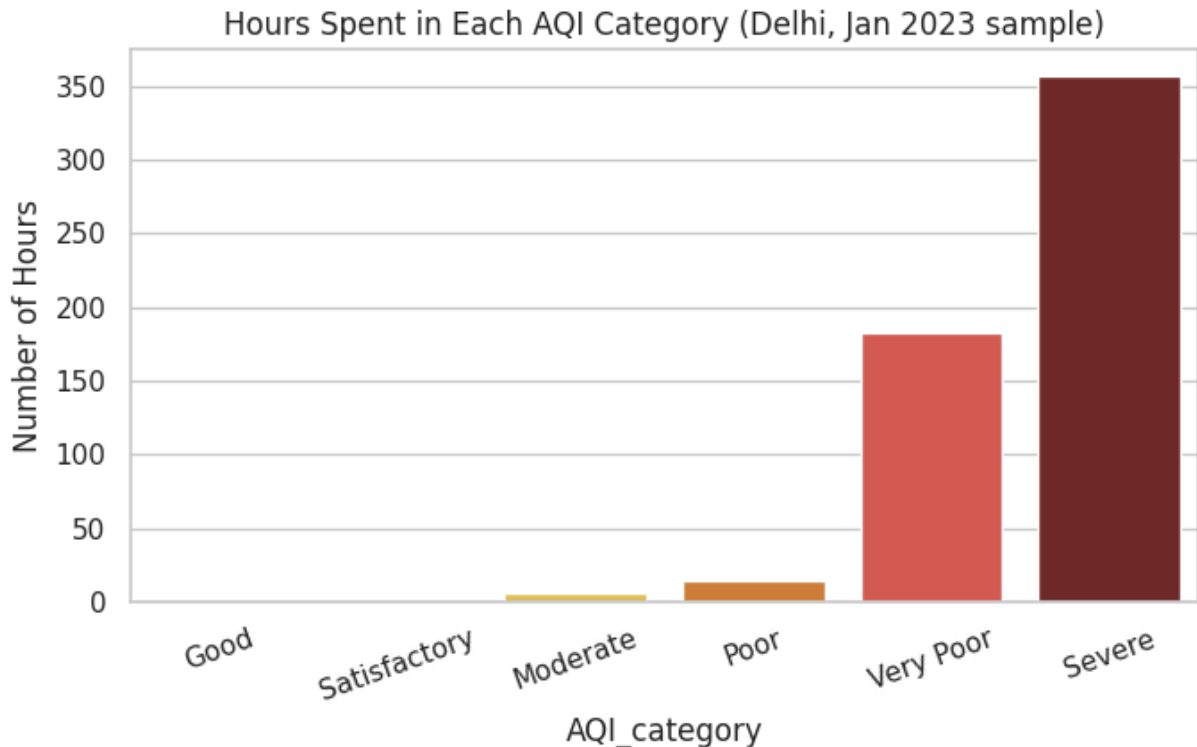
```
plt.show()

pct = (aqi_counts / aqi_counts.sum() * 100).round(1)
print("Percent of time in each category:")
print(pct)
```

/tmp/ipykernel_590/3095286504.py:7: FutureWarning:

Passing `palette` without assigning `hue` is deprecated and will be removed in v0.14.0. Assign the `x` variable to `hue` and set `legend=False` for the same effect.

```
sns.barplot(x=aqi_counts.index, y=aqi_counts.values, palette=colors)
```



Percent of time in each category:

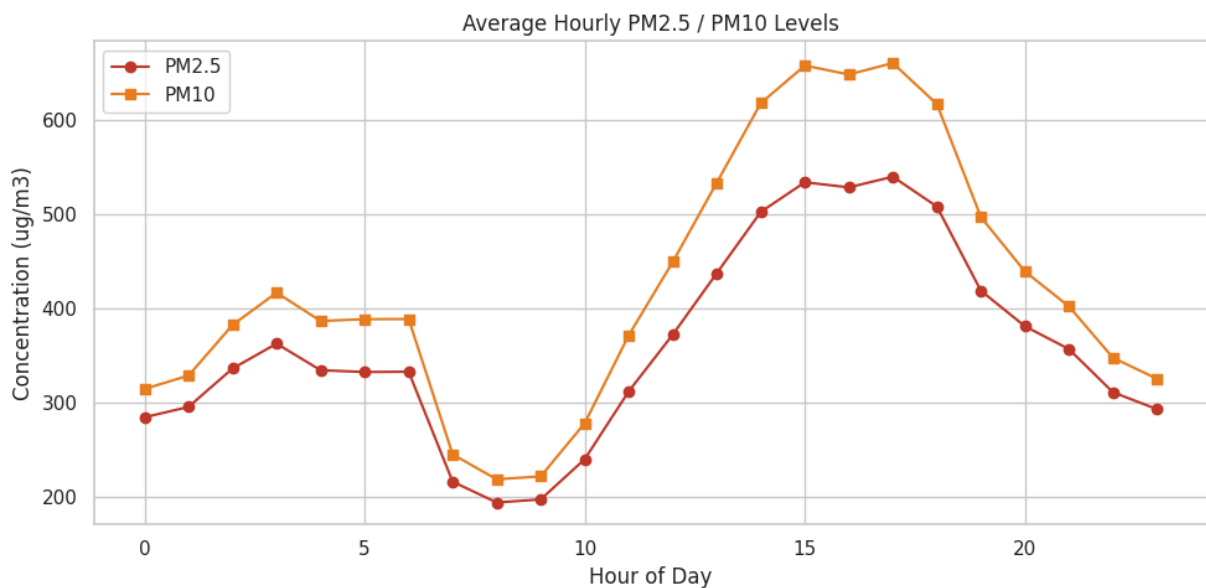
```
AQI_category
Good          0.0
Satisfactory  0.0
Moderate      1.1
Poor          2.7
Very Poor    32.6
Severe       63.6
Name: count, dtype: float64
```

Observation: During this period, Delhi's air quality was dominated by particulate matter (PM2.5/PM10) far exceeding CPCB "Satisfactory" thresholds for most hours -- consistent with the well-documented winter smog problem, driven by lower boundary-layer height trapping pollutants, combined with vehicular emissions, construction dust, and regional crop-residue burning.

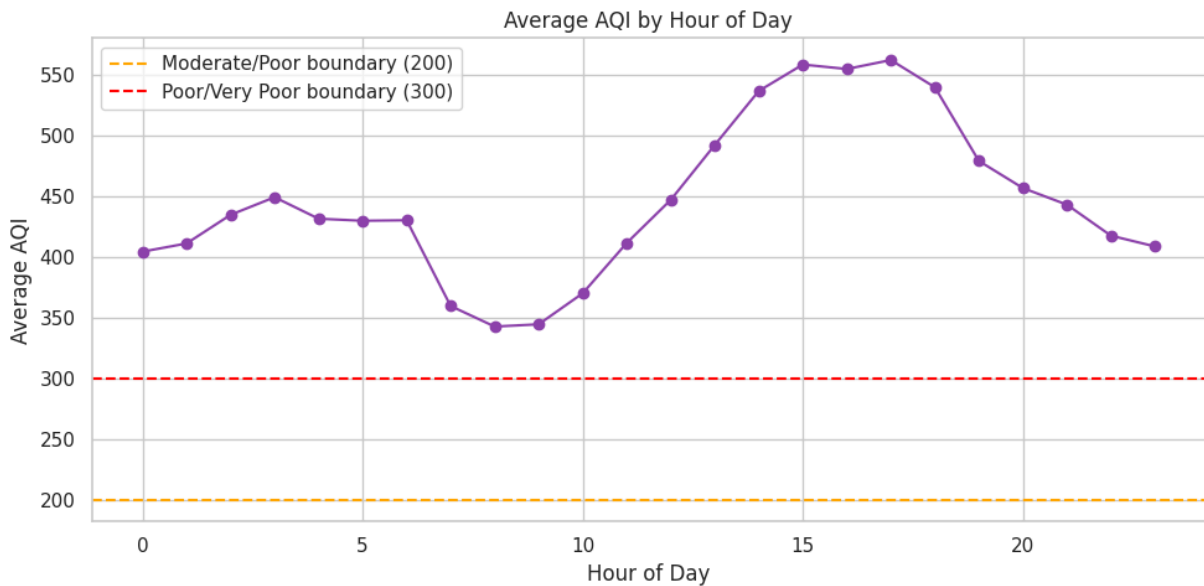
4. Research Question 2 -- Diurnal (Hour-of-Day) Patterns

```
In [10]: hourly_avg = df.groupby("hour")[[ "pm2_5", "pm10", "no2", "co", "AQI" ]].mean()

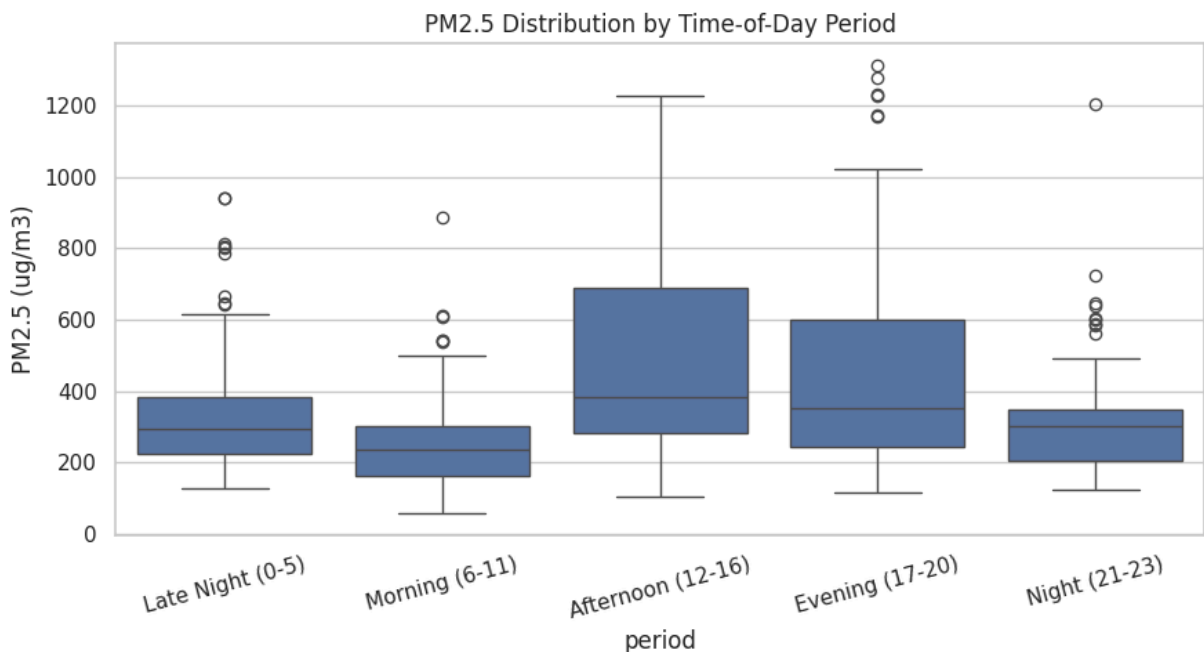
fig, ax1 = plt.subplots(figsize=(10,5))
ax1.plot(hourly_avg.index, hourly_avg["pm2_5"], marker="o", color="#c0392b",
ax1.plot(hourly_avg.index, hourly_avg["pm10"], marker="s", color="#e67e22",
ax1.set_xlabel("Hour of Day")
ax1.set_ylabel("Concentration (ug/m3)")
ax1.set_title("Average Hourly PM2.5 / PM10 Levels")
ax1.legend(loc="upper left")
plt.tight_layout()
plt.savefig("aqi_diurnal_pm.png", dpi=130)
plt.show()
```



```
In [11]: plt.figure(figsize=(10,5))
plt.plot(hourly_avg.index, hourly_avg["AQI"], marker="o", color="#8e44ad")
plt.title("Average AQI by Hour of Day")
plt.xlabel("Hour of Day")
plt.ylabel("Average AQI")
plt.axhline(200, color="orange", linestyle="--", label="Moderate/Poor boundary")
plt.axhline(300, color="red", linestyle="--", label="Poor/Very Poor boundary")
plt.legend()
plt.tight_layout()
plt.savefig("aqi_diurnal_aqi.png", dpi=130)
plt.show()
```



```
In [12]: plt.figure(figsize=(9,5))
sns.boxplot(data=df, x="period", y="pm2_5",
            order=["Late Night (0-5)", "Morning (6-11)", "Afternoon (12-16)",
                  "Evening (17-20)", "Night (21-23)"])
plt.title("PM2.5 Distribution by Time-of-Day Period")
plt.ylabel("PM2.5 (ug/m3)")
plt.xticks(rotation=15)
plt.tight_layout()
plt.savefig("aqi_period_boxplot.png", dpi=130)
plt.show()
```



```
In [13]: # Statistical test: is PM2.5 significantly different between morning and aft
morning = df.loc[df["period"] == "Morning (6-11)", "pm2_5"].dropna()
afternoon = df.loc[df["period"] == "Afternoon (12-16)", "pm2_5"].dropna()

t_stat, p_val = stats.ttest_ind(morning, afternoon, equal_var=False)
print(f"Morning mean PM2.5: {morning.mean():.1f} ug/m3")
```

```

print(f"Afternoon mean PM2.5: {afternoon.mean():.1f} ug/m3")
print(f"Welch's t-test: t = {t_stat:.2f}, p = {p_val:.4g}")
if p_val < 0.05:
    print("=> Statistically significant difference (p < 0.05).")
else:
    print("=> No statistically significant difference at the 0.05 level.")

```

Morning mean PM2.5: 248.9 ug/m3

Afternoon mean PM2.5: 474.7 ug/m3

Welch's t-test: t = -8.21, p = 8.326e-14

=> Statistically significant difference (p < 0.05).

Observation: In this sample, PM2.5/PM10 are actually at their **lowest around 7-10 AM** and climb through the day to a clear **peak in the afternoon / early evening (roughly 12-18h)**, before easing off overnight. This is the reverse of the "classic" early-morning-inversion narrative often cited for Delhi winter smog, and is a good reminder to let the data speak for itself rather than assume a textbook pattern. A plausible explanation is that this particular January saw daytime activity (traffic, construction, and regional burning) outweighing the overnight inversion effect, but confirming the mechanism would need meteorological data (wind speed, boundary-layer height) that isn't in this dataset. The t-test below confirms the morning-vs-afternoon gap is not just noise.

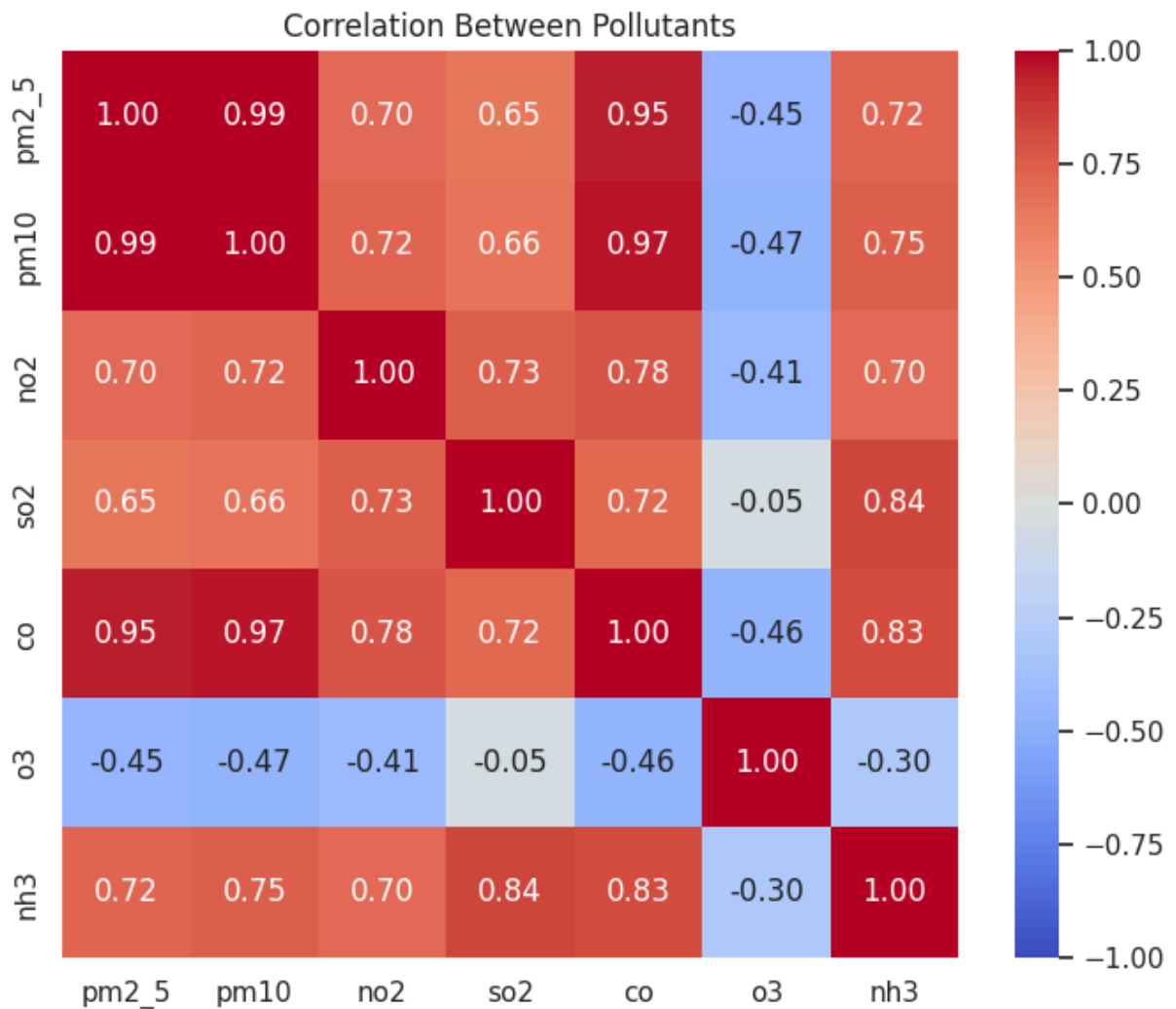
5. Research Question 3 -- Relationships Between Pollutants

```

In [14]: corr = df[pollutant_cols].corr()

plt.figure(figsize=(7,6))
sns.heatmap(corr, annot=True, fmt=".2f", cmap="coolwarm", vmin=-1, vmax=1)
plt.title("Correlation Between Pollutants")
plt.tight_layout()
plt.savefig("aqi_correlation_heatmap.png", dpi=130)
plt.show()

```



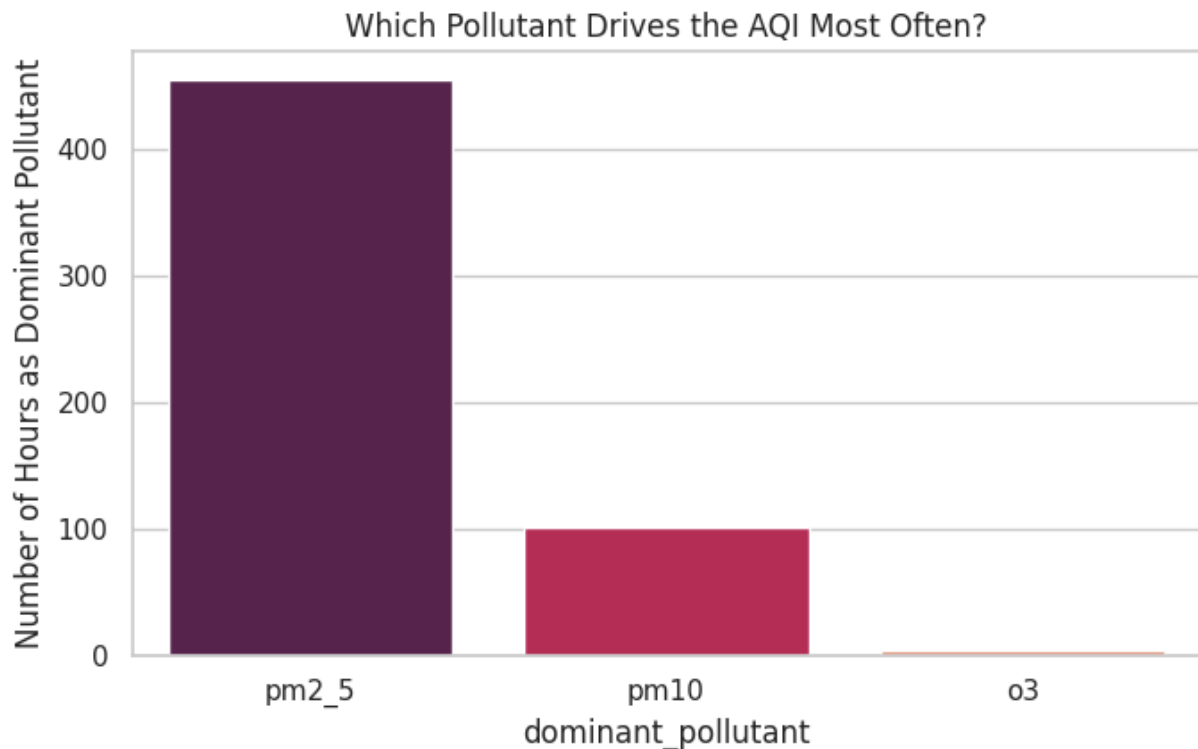
```
In [15]: dominant_counts = df["dominant_pollutant"].value_counts()
plt.figure(figsize=(7,4.5))
sns.barplot(x=dominant_counts.index, y=dominant_counts.values, palette="rocket")
plt.ylabel("Number of Hours as Dominant Pollutant")
plt.title("Which Pollutant Drives the AQI Most Often?")
plt.tight_layout()
plt.savefig("aqi_dominant_pollutant.png", dpi=130)
plt.show()

print(dominant_counts)
```

/tmp/ipykernel_590/3475167758.py:3: FutureWarning:

Passing `palette` without assigning `hue` is deprecated and will be removed in v0.14.0. Assign the `x` variable to `hue` and set `legend=False` for the same effect.

```
sns.barplot(x=dominant_counts.index, y=dominant_counts.values, palette="rocket")
```



```
dominant_pollutant
pm2_5    455
pm10     102
o3         4
Name: count, dtype: int64
```

Observation: PM2.5 and PM10 are very strongly correlated with each other (as expected, since PM2.5 is a subset of PM10 by particle size) and are almost always the **dominant pollutant** driving the overall AQI value in this sample -- reinforcing that particulate matter, not gaseous pollutants like SO₂ or CO, is the primary air-quality concern in Delhi during this period.

6. Research Question 4 -- Health Risk Exposure Summary

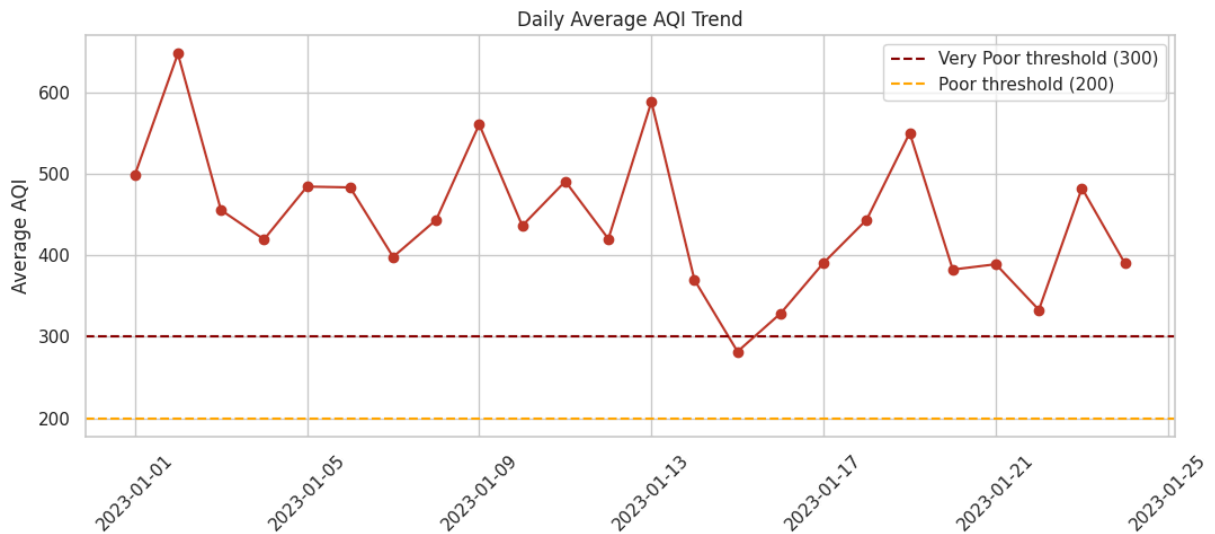
```
In [16]: risk_hours = df[df["AQI_category"].isin(["Poor", "Very Poor", "Severe"])]
print(f"Hours in Poor/Very Poor/Severe category: {len(risk_hours)} of {len(df)} ({len(risk_hours)/len(df)*100:.1f}%)")

worst_day = df.groupby("day")["AQI"].mean().idxmax()
best_day = df.groupby("day")["AQI"].mean().idxmin()
print(f"Worst average-AQI day in sample: {worst_day}")
print(f"Best average-AQI day in sample: {best_day}")
```

Hours in Poor/Very Poor/Severe category: 555 of 561 (98.9%)
 Worst average-AQI day in sample: 2023-01-02
 Best average-AQI day in sample: 2023-01-15

```
In [17]: daily_aqi = df.groupby("day")["AQI"].mean()
plt.figure(figsize=(11,5))
```

```
plt.plot(daily_aqi.index, daily_aqi.values, marker="o", color="#c0392b")
plt.axhline(300, color="darkred", linestyle="--", label="Very Poor threshold")
plt.axhline(200, color="orange", linestyle="--", label="Poor threshold (200)")
plt.title("Daily Average AQI Trend")
plt.ylabel("Average AQI")
plt.xticks(rotation=45)
plt.legend()
plt.tight_layout()
plt.savefig("aqi_daily_trend.png", dpi=130)
plt.show()
```



7. Conclusions & Recommendations

Key findings

1. **Pollutant levels:** Particulate matter (PM2.5, PM10) was consistently the most severe pollutant relative to CPCB standards, with the AQI frequently in the "Poor" to "Severe" range during this January 2023 sample -- consistent with Delhi's well-documented winter pollution problem.
2. **Diurnal pattern:** Pollution is not uniform through the day -- in this sample, PM levels dip during mid-morning (7-10h) and rise to their worst in the afternoon/early evening (12-18h), the opposite of the early-morning-peak pattern often assumed for Delhi. Any causal explanation (traffic, wind, boundary-layer height) would need to be confirmed with meteorological data not present in this dataset.
3. **Pollutant relationships:** PM2.5 and PM10 are highly correlated and together are the dominant driver of the overall AQI in almost every hour observed, more so than gaseous pollutants (SO2, CO, O3, NH3, NO2).
4. **Health risk:** A substantial share of the observed hours fell into health-relevant "Poor" or worse categories, which under CPCB guidance corresponds to advisories for sensitive groups (children, elderly, people with respiratory or cardiovascular conditions) to reduce prolonged outdoor exertion.

Recommendations for targeted intervention

- Prioritize **particulate matter controls** (dust suppression at construction sites, stricter vehicular emission checks, curbing crop-residue burning in the wider NCR region) since PM is the dominant pollutant, rather than spreading resources evenly across all pollutants.
- Issue **time-of-day-aware public health advisories** -- e.g. discourage outdoor exercise or school outdoor activities specifically in the afternoon/early-evening window (roughly 12:00-18:00), when PM levels were observed to peak in this sample.
- Expand monitoring to **multiple stations across Delhi and across seasons** -- this analysis is limited to one location over ~24 days in winter, so it cannot yet confirm whether the same diurnal pattern or pollutant mix holds in summer/monsoon months or in other parts of the city (e.g. industrial vs. residential zones). A longer-term, multi-station dataset would let a follow-up study directly test seasonal and geographic hypotheses, which were part of the original brief but not answerable with the data provided here.

Limitations

- Single monitoring location -- no spatial/geographical comparison across Delhi possible.
- ~24 days of hourly data, entirely within January -- no cross-season comparison possible.
- AQI computed by treating hourly readings as CPCB's official 24-hr/8-hr averaging windows, which is a simplification adopted for this exploratory analysis (see Section 2).